

1^{re} - NSI - FICHE 5.1 - TYPES ET VALEURS DE BASE
REPRÉSENTATION DES NOMBRES : LES ENTIERS NATURELS

Dans un ordinateur, toutes les données (images, textes, vidéos, nombres...) sont transmises et stockées à l'aide de composants électroniques : les transistors. Ceux-ci ne peuvent se trouver que dans 2 états : sous tension (état 1) ou hors tension (état 0).

Ainsi, en informatique l'unité de mesure de base est le bit (binary digit) qui ne peut prendre que 2 valeurs 0 ou 1. En informatique, toute action est une suite d'opérations sur des paquets de 0 et de 1, regroupés par 8 : les octets. Pour information, en 1971, le premier microprocesseur commercialisé comptait 2300 transistors. En 2018, le microprocesseur IntelCore i7 pour le grand public en compte 2,3 milliards. Actuellement, le microprocesseur Apple M3 Ultra comprend 184 milliards de transistors.

Les processeurs récents sont des processeurs 32 ou 64 bits : ils disposent de 32 ou 64 bits pour stocker un nombre.

I. REPRESENTATION BINAIRE D'UN NOMBRE ENTIER NATUREL

1. La base 10 ou le système décimal

Depuis la maternelle vous avez appris à compter en utilisant le système décimal, et sans forcément le savoir, vous avez compté en base 10. Cela signifie que "vous avez fait des paquets de 10" mais aussi que :

- vous n'avez utilisé que 10 symboles, appelés chiffres : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- le rang dans le nombre de chaque chiffre représente une puissance de 10.

Exemple : 587 est un nombre composé de 3 chiffres.

7 est le chiffre des unités et une unité est égale à 10^0 , 8 est le chiffre des dizaines et une dizaines est égale à 10^1 , 5 est le chiffre des centaines et une centaines est égale à 10^2 .

Ainsi : $587 = 5 \times 10^2 + 8 \times 10^1 + 7 \times 10^0$

Cette décomposition est appelée décomposition en puissance de 10.

Exercices :

a. Écrire de la même manière que précédemment $5084 = 5 \times 10^3 + 0 \times 10^2 + 8 \times 10^1 + 4 \times 10^0$

b. Donner l'écriture décimale de :

☞ $5 \times 10^5 + 3 \times 10^1 + 4 = 500034$

☞ $3 \times 10^4 + 2 \times 10^3 + 3 \times 10^2 + 5 \times 10^1 = 32350$

2. La base 2

Le fonctionnement d'un ordinateur est lié à des transistors qui peuvent se trouver dans 2 états :

- l'état 1 : le courant électrique passe ;
- l'état 0 : le courant électrique ne passe pas.

D'où l'idée de représenter les informations, en particulier les nombres, qui seront manipulées par un ordinateur en une suite de 0 et de 1.

Par analogie avec le système décimal, dans le système binaire, on compte en base 2, on fait des paquets de 2 et :

- on n'utilise que 2 symboles, appelés **bits** : 0 et 1.
- le rang de chaque bit représente une puissance de 2.

bit : contraction de "binary digit"
littéralement "chiffre binaire"

Exemple : 1101 est l'écriture ou **encodage binaire** du nombre $1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13$
décimal 8 + 4 + 0 + 1 = 13

Remarque : 1101 utilise 4 bits . On dit que 1101 est l'encodage en binaire de 13 sur 4 bits.

Quelques puissances de 2 à connaître :

$2^0 = 1$, $2^1 = 2$, $2^3 = 8$, $2^4 = 16$, $2^5 = 32$, $2^6 = 64$, $2^7 = 128$, $2^8 = 256$, $2^9 = 512$, $2^{10} = 1024$, $2^{11} = 2048$

Attention : 1010 en base 10, ce n'est pas le même nombre que 1010 en base 2!

Pour éviter les confusions on précisera la base de la manière suivante :

- ☞ lorsqu'il s'agit du nombre décimal : $(1010)_{10}$
- ☞ lorsqu'il s'agit du nombre binaire $(1010)_2$ ou encore $b1010$

a. Trouver la représentation en base 2 d'un entier naturel donné en base 10.

Première méthode : on fait des "paquets" de 2 : cela revient à faire des divisions euclidiennes par 2.

Divisions par 2	Ecriture binaire
$157 = 78 \times 2 + 1$	 1
$78 = 39 \times 2 + 0$	 0 1
$39 = 19 \times 2 + 1$	 1 0 1
$19 = 9 \times 2 + 1$	 1 1 0 1
$9 = 4 \times 2 + 1$	... 1 1 1 0 1
$4 = 2 \times 2 + 0$	. 0 0 1 1 1 0 1
$2 = 1 \times 2 + 0$	. 0 0 1 1 1 0 1
$1 = 0 \times 2 + 1$	1 0 0 1 1 1 0 1
$0 = 0 \times 2 + 0$	1 0 0 1 1 1 0 1

En base 2, 157 s'écrit : $b10011101$.

Pour écrire 157, on a besoin de 8 bits. On ne peut pas coder 157 sur 4 bits par exemple. Il y a problème d'overflow.

On peut écrire :

$$157 = 1 \times 2^0 + 0 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 + 1 \times 2^4 + 1 \times 2^7.$$

Dans la notation binaire, le bit le plus à gauche est appelé bit de poids fort ; le bit le plus à droite est appelé bit de poids faible.

Autre présentation : à l'aide des divisions successives par 2 :

Handwritten division steps for 157 by 2, showing the sequence of remainders (1, 0, 1, 1, 1, 0, 0, 1) which read from bottom to top gives the binary representation 10011101.

Deuxième méthode : on décompose le nombre en une somme de puissances de 2 :

	$128 = 2^7$	$64 = 2^6$	$32 = 2^5$	$16 = 2^4$	$8 = 2^3$	$4 = 2^2$	$2 = 2^1$	$1 = 2^0$
157 =	1	0	0	1	1	1	0	1

Ainsi : $157 = 128 + 16 + 4 + 1 \rightarrow 10011101$

<https://www.youtube.com/watch?v=ByVoCc57PwM&t=21s>



b. Conversion d'un nombre binaire en décimal.

On utilise les puissances de 2 :

$$b10100011 = 1 \times 2^0 + 1 \times 2^1 + 0 \times 2^2 + 0 \times 2^3 + 0 \times 2^4 + 1 \times 2^5 + 0 \times 2^6 + 1 \times 2^7 = 163$$

$$b11111111 = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 = 255$$

3. En machine

- Avec une mémoire de 4 bits, on peut coder les nombres de 0000 à 1111 soit $2^4 = 16$ nombres compris entre 0 à 15. Ou encore, on peut écrire les nombres de 0 à $2^4 - 1$.
- Avec une mémoire de 8 bits, $2^8 \rightarrow 256$ nombres de 0 à 255
- Avec une mémoire de 16 bits, $2^{16} \rightarrow$ de 0 à $2^{16} - 1$
- Avec une mémoire de 32 bits, $2^{32} \rightarrow$ de 0 à $2^{32} - 1$
- Avec une mémoire de 64 bits, $2^{64} \rightarrow$ de 0 à $2^{64} - 1$

- Avec une mémoire de n bits, on peut écrire 2^n nombres qui vont de 0 à $2^n - 1$

4. Addition de deux nombres binaires

Les ordinateurs ne connaissent pratiquement qu'une seule opération : l'addition.

Pour additionner 2 nombres en base 2, on procède comme en base 10 mais en ayant toujours en mémoire que $b1+b1=b10$.

$$\begin{array}{r} 1101 \\ + 1001 \\ \hline 10110 \end{array}$$

Ainsi en additionnant $13+9$, on obtient :

Attention : la mémoire d'un ordinateur est organisée en "cases" de telle manière qu'on puisse y stocker des mots binaires (suite de 0 et de 1) de taille fixe.

Les premiers ordinateurs avaient une mémoire de 8 bits : une case mémoire permettait de stocker un mot de 8 bits c'est à dire un octet.

Après être passé à 16 bits, puis 32 bits la plupart des ordinateurs ont maintenant une mémoire 64 bits.

Exemple : Avec une mémoire 8 bits . 199 est représenté dans une "case" mémoire par l'octet : 11000111
64 est représenté dans une "case" mémoire par l'octet : 01000000

Mais $1100\ 0111 + 0100\ 0000 = 1\ 0000\ 0111$ (9 bits) ne peut être représenté sur un octet . Le bit de poids fort (le plus à gauche !) est perdu !

Ainsi pour une mémoire 8 bits, $1100\ 0111 + 0100\ 0000 = 0000\ 0111$ (le 1 le plus à gauche est perdu) ce qui signifie que pour une machine 8 bits $199 + 64 = 7!!$

On dit qu'il y a **overflow**.

Remarque : le bit le plus à gauche s'appelle le bit de poids fort, alors que celui le plus à droite est le bit de poids faible.

EXERCICES :

- Convertir les nombres suivants en binaire : 458 ; 133 ; 47 ; 1024 ; 65
- Convertir les nombres suivants écrits en binaire en décimal : 101010101 ; 111000 ; 00110011 ; 1010010001

II. L'ÉCRITURE HEXADÉCIMALE.

L'écriture binaire est peu lisible et un peu longue à écrire. Pour plus de facilité, on utilise la base 16 (hexadécimale). Cette écriture utilise les chiffres de 0 à 9 et les lettres A (10), B (11), C (12), D (13), E (14) et F (15). On fait en base 16 des paquets de 16. Les nombres écrits en hexadécimal sont précédés de la lettre x.

Décimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Binaire	0	1	10	11	100	101	110	111	1000	1001	1010	1011	1100	1101	1110	1111	10000
Hexadécimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	x10

1. Trouver la représentation en base 16 d'un entier naturel donné en base 10.

On fait des divisions par 16.

Divisions par 16	Ecriture hexadécimale
$965 = 60 \times 16 + 5$	 5
$60 = 3 \times 16 + 12$	 c 5
$3 = 0 \times 16 + 3$	 3 c 5

En hexadécimal, 965 s'écrit : x3c5

<https://www.youtube.com/watch?v=ZDbvNzxPPXI&t=142s>



2. Convertir un hexadécimal en décimal.

$$xC25 = 12 \times 16^2 + 2 \times 16 + 5 = 3109$$

<https://www.youtube.com/watch?v=jEeptIOD014&t=25s>



3. Convertir un nombre binaire en hexadécimal.

On le découpe en tranches de 4 chiffre. ($2^4 = 16$, $2^8 = 16^2$, $2^{12} = 16^3$...).

$$b100101110 = 1 \times 2^8 + b0010 \times 2^4 + b1110 \times 2^0 = 1 \times 16^2 + 2 \times 16^1 + 14 \times 16^0 = x14E$$

4. Convertir un hexadécimal en binaire.

$$xE28 = E \times 16^2 + 2 \times 16^1 + 8 \times 16^0 = E \times 2^8 + 2 \times 2^4 + x8 \times 2^0 = b111000101000$$

<https://www.youtube.com/watch?v=2TDTZQmcyWE&t=22s>



EXERCICES :

- Convertir les nombres binaires suivants en hexadécimal : 10000000011001 ; 1000100010001 ; 100110000111 ; 1100101011111100010 cage2
- Convertir en hexadécimal les nombres suivants : 2023 ; 1234 ; 56026 ; 64218
- Convertir en binaire les nombres suivants écrits en hexadécimal : A320 ; FABE51 ; 101010 ; 59A75

III. EXERCICES

1. Vrai/Faux ?

- ☒ a. Si l'écriture binaire d'un entier se termine par n zéros, alors cet entier est divisible par 2^n .
- ☐ b. En base 8, on utilise les chiffres de 1 à 8 *de 0 à 7.*
- ☒ c. Avec n bits, les entiers représentables sont strictement inférieurs à 2^n .
- ☒ d. 170 s'écrit AA en hexadécimal.

2. QCM Pour chaque question, une seule réponse est exacte parmi celles proposées.

- a. Le système binaire repose sur les chiffres a. 0 et 1 b. du système décimal c. 1 et 2
- b. Le nombre de caractères du système hexadécimal est : a. 14 b. 15 c. 16 d. 17
- c. On considère 1000 écrit en base 10. Quelle affirmation est exacte ?
- ~~i.~~ Ce nombre s'écrit AAA en hexadécimal.
- ii. Ce nombre s'écrit avec 9 chiffres en binaire.
- ~~iii.~~ Ce nombre s'écrit avec 4 chiffres en hexadécimal.
- iv. L'écriture de ce nombre en binaire se termine par 000
- d. Quelle affirmation est exacte ?
- i. Un nombre écrit en hexadécimal comporte 8 fois moins de chiffres que s'il est écrit en binaire.
- ii. Un nombre pair a une écriture binaire qui se termine par un 0.
- iii. Un nombre pair a une écriture hexadécimale qui se termine par un 0.
- e. Quelle est la valeur en binaire de 1001×111 (une multiplication en binaire se pose comme en décimal) : a. 111111 b. 101010 c. 100111 d. 111001

3. Les règles de communication sur internet sont définies par un protocole universel : IP (Internet Protocol). Actuellement deux versions de ce protocole co-existent : l'IP V4 (version 4, voué à disparaître) et l'IP V6 (Version 6).

- Avec IPV4 chaque machine connectée à internet est identifiée sur ce réseau par une adresse unique constituée de 4 nombres compris entre 0 et 255 en décimal séparés par un point .

Exemple : 158.1.156.58

- Avec IPV6 chaque machine reçoit une adresse constituée de 8 nombres compris entre 0 et ffff en hexadécimal , séparés par deux points.

Exemple : a015 :7895 :a000 :0 :ab00 :1 :d :78f0

- a. Combien de bits sont nécessaires pour encoder en binaire une adresse IPV4 ? Une adresse IPV6 ? *32 bits 128 bits*
- b. En prenant pour approximation $2^{10} = 1024 \simeq 10^3$ donner une approximation du nombre d'adresses IPV4 et du nombre d'adresses IPV6 possibles.

4. Les couleurs RVB sont représentées par un triplet (R,V,B) de trois nombres entiers naturels compris entre 0 et 255, où R correspond à l'intensité de rouge, V l'intensité vert et B l'intensité de bleu. Une telle représentation est appelée code ASCII.

- a. De combien d'octets a-t-on besoin pour représenter une couleur RVB ? *24*
- b. La couleur marron a pour code ASCII (88,41,0). Quel est son code binaire ? *01011000 00101001 00000000*
- c. La couleur turquoise a pour code ASCII (37,253,233). Quel est son code hexadécimal ? *# 25FDE9*
- d. La couleur orange a pour code hexadécimal ED7F10. Quels sont ses codes binaire et ASCII ?
- e. La couleur rouge carmin a pour code binaire 100101010000000000011000. Quels sont ses codes hexadécimal et ASCII ? *226*

IV. TP : CONVERSION DE NOMBRES (BINAIRE EN DÉCIMAL ET DÉCIMAL EN BINAIRE)

Voir CAPYTALE, code : b135-1182630

UN PEU D'AIDE :

EXERCICE 1

Le but de cette fonction est de convertir un nombre écrit sous forme binaire en décimal.

Aide 1 : observez : $1111011 = 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 123$

Aide 2 : le nombre binaire est écrit sous forme de chaîne de caractères. Il faut parcourir cette chaîne de caractères.

Aide 3 : le premier caractère d'une chaîne de caractère a pour indice 0. Donc si $x = '1111011'$, $x[4] = '0'$

Aide 4 : on doit faire une somme. Il faut donc initialiser une variable somme à 0 et au fur et à mesure lui rajouter 1×2^6 , puis 1×2^5 ... Donc écrire $\text{somme} = \text{somme} + \text{"expression en fonction de la puissance de 2"}$.

Aide 5 : pour parcourir la chaîne de caractère, on utilise : `for i in range(len(x))`. `i` prend toutes les valeurs de 0 à $\text{len}(x)-1$. Soit au total $\text{len}(x)$ valeurs.

Aide 6 : $x[i]$ est un caractère, pour effectuer des calculs, il faut le convertir en entier. On obtient alors 0 ou 1 (chiffres).

Aide 7 : quel est l'exposant de 2 en fonction de $\text{len}(x)$ et `i` ?

EXERCICE 2

Aide 1 : compléter les différentes étapes permettant de donner l'écriture binaire du nombre 133 :

Étape	n en décimal	Reste de la division euclidienne de n par 2	Bits de l'écriture binaire	Quotient de la division euclidienne de n par 2
1	133	1	'1'	66
2	66	0	'01'	33

On continue ce procédé **tant que**

Aide 2 : ne pas oublier de convertir le reste en str.

Aide 3 : il faut au fur et à mesure rajouter des 0 ou des 1 à une chaîne de caractère vide au départ. Voir l'exercice sur l'écriture d'un mot à l'envers.

Aide 4 : à chaque tour de boucle, on remplace la valeur du nombre n par ...